BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 267 · Interrupts and service routines

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Define fetch-decode-execute.

Q2. What causes a process to enter Waiting?

Q3. What does an OS scheduler choose?

Q4. Simplify (A∧B)∨(A∧¬B), showing factoring.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

interrupt — CPU attention request

ISR — Interrupt Service Routine

context — State for safe resume

polling — Repeatedly checking event state

New learning

Understand — An interrupt signals an event such as device completion to the processor.

Trace — The CPU saves sufficient execution context before servicing the event.

Explain — An ISR handles the interrupt, acknowledges it, then restores normal execution state.

Worked example

Keyboard triggers an interrupt during a text editor. Trace the stages.

1. Predict first — The ISR processes keyboard input before restoring the prior program state.

We Do · Practise with guidance

A printer signals I/O completion. Describe ISR and scheduler actions.

```
Printer complete → interrupt → ISR acknowledges → spool next job
```

Predict first

The ISR can acknowledge completion and allow queued device work to proceed; the interrupted task resumes.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Define interrupt, ISR and one source of interrupts. [4]

Question 2

Why does CPU context need to be saved/restored around a device ISR? [6]

Question 3

Compare polling and interrupt-driven I/O for infrequent device input. [6]

Exit Ticket · Answer before checking

An interrupt handler can overwrite original registers without preserving any state.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Read, interpret, execute an instruction.

Q2. Unavailable I/O or required event.

Q3. Which Ready process receives CPU time.

Q4. $A \wedge (B \vee \neg B) = A \wedge 1 = A$.

You Do pages · Independent questions

Q1. Interrupt signals attention [1], ISR is code handling event [2], e.g. keyboard or timer [1].

Q2. Interrupt arrives [1]; preserve PC/required registers [2]; ISR handles event [1]; restore [1]; correct original program continuation [1].

Q3. Polling repeatedly checks [2], wastes cycles [1]; interrupt-driven signals events [1], frees CPU for other work [1], adds ISR/context overhead [1].

Exit Ticket · Model answer

False. Necessary state must be saved and restored.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: